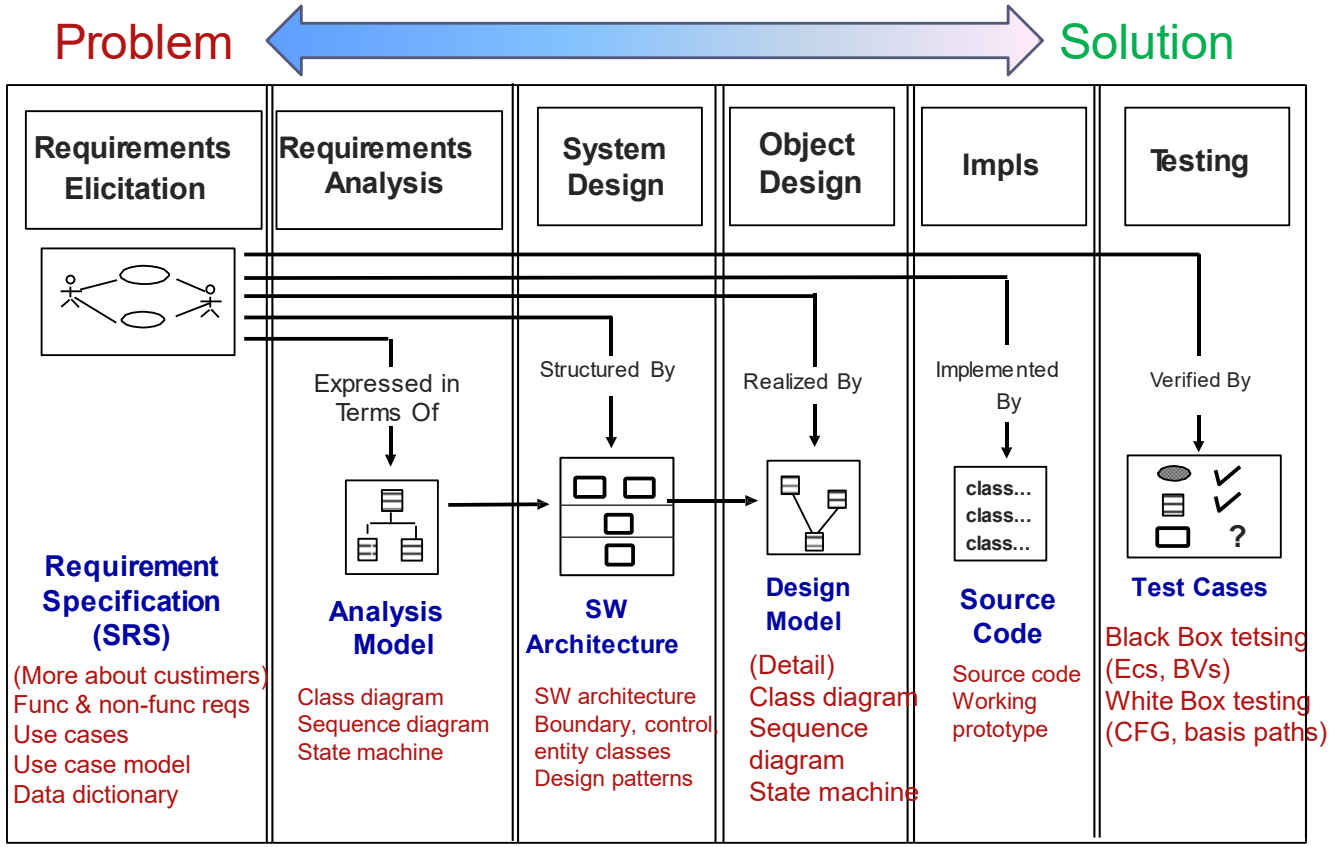
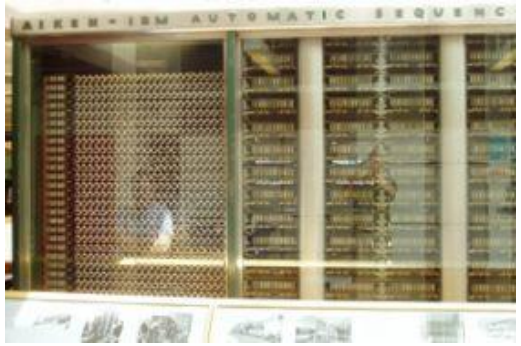
Lecture 16: Test Fundamentals

Liu Yang



SDLC Activities

What is a computer bug?



- In 1947 Harvard University was operating a room-sized computer called the Mark II.
  - mechanical relays
  - glowing vacuum tubes
  - technicians program the computer by reconfiguring it
  - Technicians had to change the occasional vacuum tube.
- A moth flew into the computer and was zapped by the high voltage when it landed on a relay.
- Hence, the first computer bug!



Unexpected things happen!

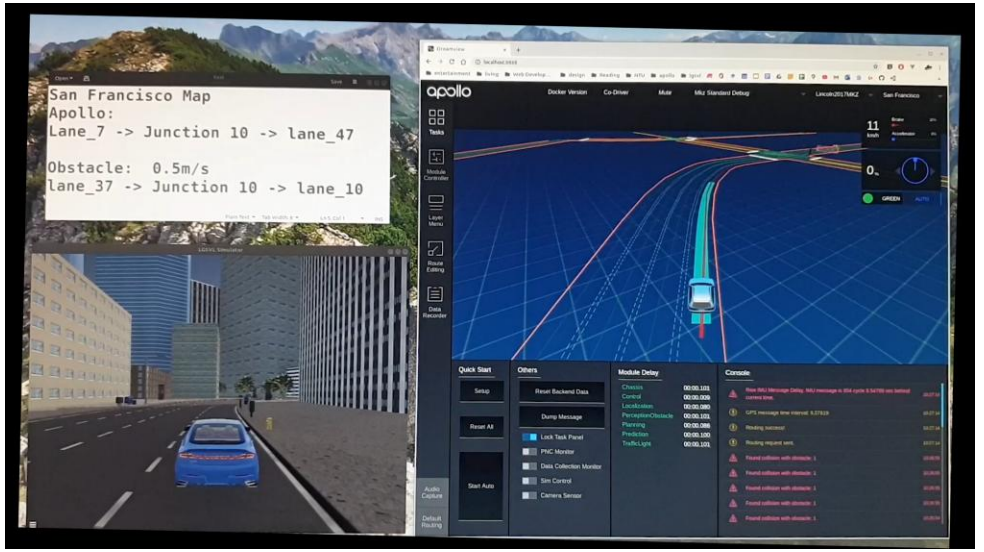


When I launch my app after several hours of development

A **software bug** is an error, flaw, failure or fault in a computer program or system that causes it to produce an incorrect or unexpected result, or to behave in unintended ways.

List of software bugs

- Space
- Medical devices
- Tracking years
- Electric power transmission
- Telecommunications
- Military
- Media
- Video gaming
- Encryption
- Transportation
- Finance
- Blockchain

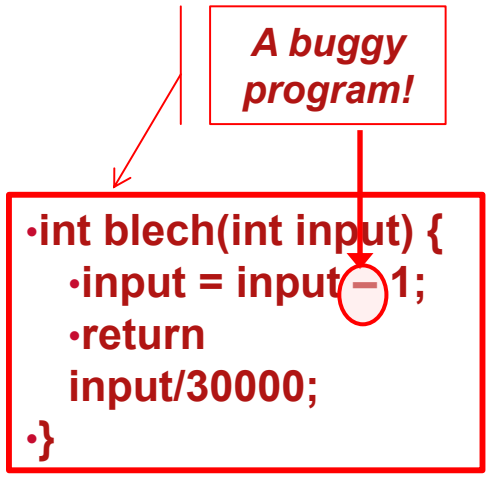


Can We Test Everything?

- A function is supposed to

Take an integer input value, add 1 to the input, divide it by 30000 and return the value

- How many possible input values for 16-bits integer?
  - $2^{16} = 65536$



Can we test all the possible inputs?

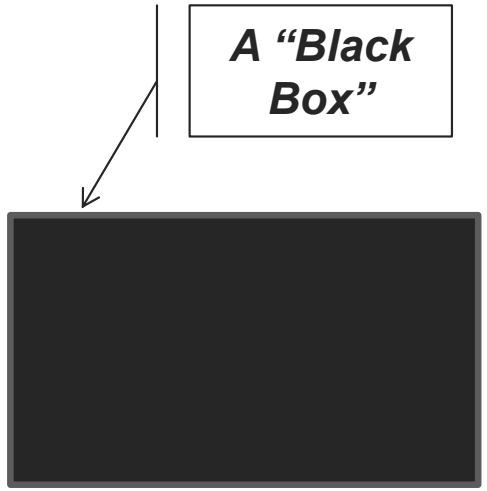
Yes, but TOO COSTLY!

Can We Test Everything?

A function is supposed to
Take an integer input value, add 1 to the input, divide it by 30000 and return the value

Which input values do you choose to test the program?

- 1, 42, 32000, -31218, …



Are these good selection of inputs?
Can we detect the bug of the program?

Equivalence Class and Boundary Value Testing

- Add1Divide30000 (-32768 ≤ input ≤ -30001) [return -1]
  - Lower boundary -32768: ~~-32769~~, -32768, ~~-32767~~
  - Upper boundary -30001: ~~-30002~~, -30001, ~~-30000~~

- Add1Divide30000 (-30000 ≤ input ≤ 29998) [return 0]
  - Lower boundary -30000: ~~-30001~~, -30000, ~~-29999~~
  - Upper boundary 29998: ~~29997~~, 29998, ~~29999~~

- Add1Divide30000 (29999 ≤ input ≤ 32767) [return 1]
  - Lower boundary 29999: ~~29998~~, 29999, ~~30000~~
  - Upper boundary 32767: ~~32766~~, 32767, 32768

Can We Test Everything?

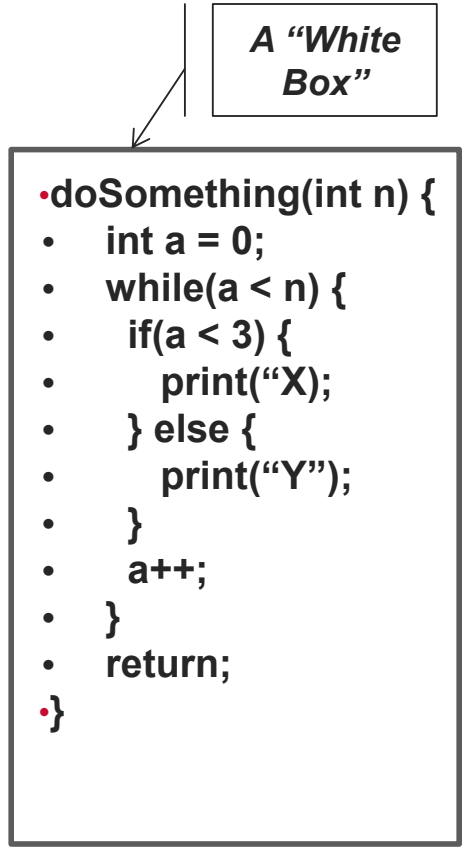
How many possible input values for 16-bits integer n?

- $-2^{16} = 65536$

Which input values do you choose to test the program?

- 1, 42, 32000, -31218, …

Can we cover all statements, all branches, or all execution paths?



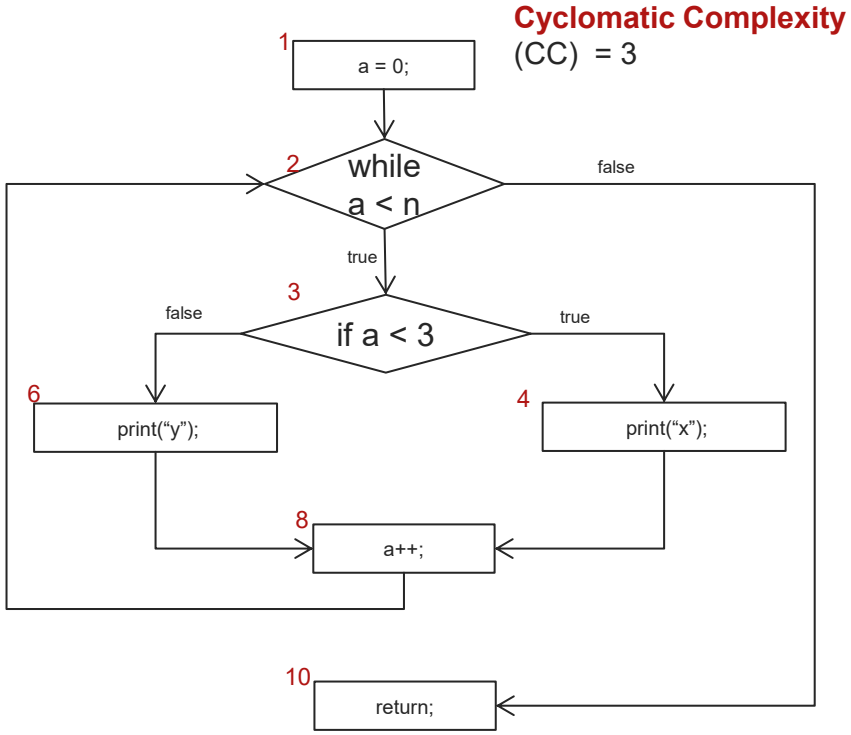
Basis Path Testing

Basis Path Testing (or structured testing) is a white box testing method used for designing test cases intended to examine all possible paths of execution in a program at least once.

- Analyzes Control flow graph (CFG)
- Determines Cyclomatic Complexity (CC) value
- Obtains linearly independent paths
- Generates test cases for each path

Basis Path Testing

Control Flow Graph (CFG)



Cyclomatic Complexity
(CC) = 3

- Three basis paths
- 1, 2, 10
- 1, 2, 3, 4, 8, 2, 10
- 1, 2, 3, 6, 8, 2, 10

- Three test cases
- n = 0
- n = 1
- n = 4

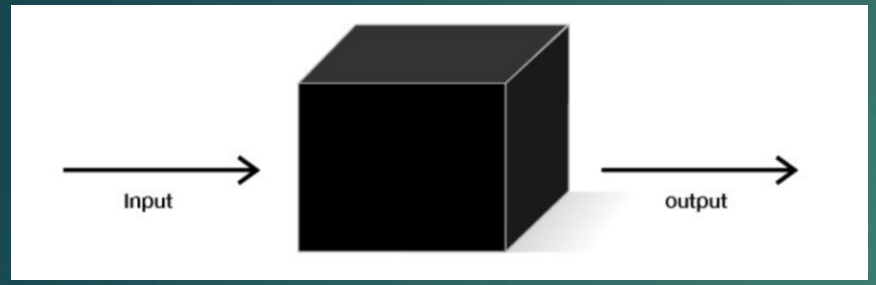
- Real execution paths
- 1, 2, 10
- 1, 2, 3, 4, 8, 2, 10
- 1, 2, 3, 4, 8, 2, 3, 4, 8, 2, 3, 4, 8, 2, 3, 6, 8, 2, 10

What is Testing?

Testing:
Software *testing can* be used to *show the presence* of *bugs, but* never to *show their absence.*
-- Edsger Dijkstra

- Testing == Debugging ?
- To show that software works ?
- To show that software doesn't work ✓
- To reduce the perceived risk of system not working to an acceptable value ✓
- To result in low-risk software without much testing effort ✓

Types Of Testing

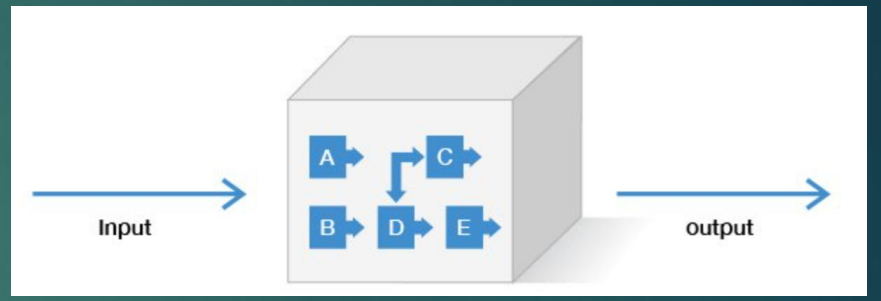


Black Box Testing

- What to analyze: Requirements and specifications
  - E.g. "*The income of an applicant must be between $1500 per month and $8000 per month*"

Type of black box testing

- **Equivalence class testing and boundary value testing**
- State-based testing



White Box Testing

- What to analyze: Implementation details, internal paths and structure

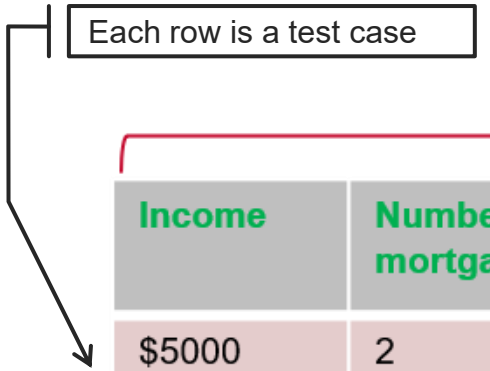
- Type of white box testing
  - **Control flow testing**
  - Data flow testing

Levels Of Testing

- Unit testing (we learn in this course)
  - Test an unit individually
- Integration testing
  - Test units in combination
- System testing
  - Test everything (functionality, usability, reliability, performance, portability, security …)
- Acceptance testing (UAT User Acceptance Testing)
  - Customer accepts the software and give us their money

Ingredients Of Test Case

Ingredient	Description
Name	Distinct name of test case
Location	Path to test program and its inputs
Input	Input data or commands
Oracle	**Expected test output**
Log	Actual output produced by the test



Input | Oracle | Log

Income	Number of mortgage	Applicant Type	Property Type	Expected Result	Test Result
$5000	2	Person	Cando	Approval	Approval
$30000	3	Person	HDB	Approval	Approval
$15000	4	Person	HDB	Approval	Approval
$100	1	Person	HDB	Reject	Reject
$1501	8	Person	Condo	Reject	Approval
$19845	3	Company	Condo	Reject	Reject
$79999	2	Person	Mobile home	Reject	Reject

Test Cases for Mortgage Application

An Android Unit Test Case

```
•package mortgageapplication.test;

•public class TestMortgageApplicationActity {
    –private MortgateProcessor processor;
    –public void testQualifiedApplication() {
    –   int income = 5000;
    –   int nummortage = 2;
    –   AppcantType applicant = ApplicantType.Person;
    –   PropertyType property = PropertyType.Cando;
    –    ApplicationResult result =
    –          processor.analyzeApplication( income,
    –                    numortgage, applicant, property);
        •assertEquals(result, ApplicationResult.Approval);
    –}
    –public void testLowIncome() {
    –   int income = 100;
    –   int nummortage = 1;
    –   AppcantType applicant = ApplicantType.Person;
    –   PropertyType property = PropertyType.HDB;
    –    ApplicationResult results=
    –          processor.analyzeApplication( income,
    –                    numortgage, applicant, property);
        •assertEquals(result, ApplicationResult.Reject);
    –}
•}
```

Location

Name

Input

Log

Oracle

Google "Android Testing Fundamentals" and "Android Activity Testing Tutorial"

Order Of Test Case Execution

- Cascading test cases, e.g.,
  - Create an order
  - Read the order
  - Update the order
  - Delete the order
  - Read the deleted order

✓ Smaller and simpler test cases

✗ One fails, the subsequent test may be invalid

- Independent test cases, e.g.,
  - Self-contained test cases for the above features

✓ Parallel test execution

✗ Larger and more complex test cases

Summary of Testing

- Software testing is an investigation conducted to provide stakeholders with information about the quality of the software product or service under test.
  - Dynamic execution
  - To find bug, rather than to prove no bug
- Type of testing
  - Whitebox vs Blackbox
- Level of testing
  - Unit, integration, system and acceptance
- Test Case
  - 5 elements, and execution order